Q1 Simple linear regression

Model obtained

```
from sklearn.linear_model import LinearRegression

X = df[["Length"]]
y = df["Diameter"]

#create the liner model
model = LinearRegression()
#fit the model to the data
model.fit(X, y)

#get model coefficient -> slope a
a = model.coef_[0]
# intercept b
b = model.intercept_

#compute correlation cofficient between length and diamter
corr = np.corrcoef(df["Length"], df["Diameter"])[0,1]

print("Model: Diameter = a X Length + b")
print("a =", a)
print("b =", b)
print("Correlation =", corr)
```

```
Model: Diameter = a X Length + b
a = 0.8154869175609554
b = -0.019413785519976612
Correlation = 0.9868115846024929
```

The fitted linear model is `Diamter = 0.8155*Length - 0.0194`
The correlation cofficient between Length and Diamter is r=0.98681

Interpretation

- The correlation coefficient is **very high** (≈0.99) meaning that there's almost an exact linear relationship between length and diameter.
- The insignificant negative intercept is biologically irrelevant (an abalone cannot have a negative diameter), is not the sum of errors but stands for measurement noise and indicates that the regression line has **been optimized along non-zero lengths**.

Conclusion

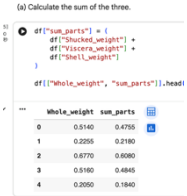
The regression coefficients show that abalone shell size **can be predicted** from length alone. This **tight linear correlation** aligns with the morphological character definitions included in the dataset documentation.

Q2 Multilinear model

1) Calculating the sum of the three component weights

Computed the sum of shucked weight, viscera weight, and shell weight for each abalone specimen `sum_parts=Shucked_weight+ Viscera_weight+Shell_weight`

(a) Calculate the sum of the three.



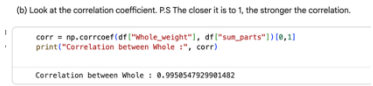
```
df["sum_parts"] = (
    df["Shucked_weight"] +
    df["Viscera_weight"] +
    df["Shell_weight"]
)

df[["Whole_weight", "sum_parts"]].head(
```

	Whole_weight	sum_parts
0	0.5140	0.4755
1	0.2255	0.2180
2	0.6770	0.6080
3	0.5160	0.4845
4	0.2050	0.1840

2) Correlation between Whole weight and the sum of its components

The correlation coefficient is r=0.99505
This indicates an extremely strong linear relationship.
The closer to 1, the stronger the correlation.

(b) Look at the correlation coefficient. P.S The closer it is to 1, the stronger the correlation.



```
corr = np.corrcoef(df["Whole_weight"], df["sum_parts"])[0,1]
print("Correlation between Whole : ", corr)
```

```
Correlation between Whole : 0.9950547929901482
```

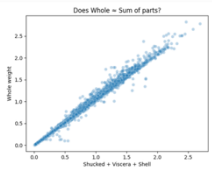
3) Difference between Whole and the sum of parts

To examine whether whole weight is exactly equal to the sum, we computed: diff=WholeWeight−sumparts

The summary statistics for this difference are Notable observations: Mean difference ≈ **0.050 grams** / Standard deviation ≈ **0.058 grams** / Minimum = –0.4475, Maximum = 0.6080 Whole weight is consistently **slightly larger** than sum_parts

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(x=df["sum_parts"], y=df["Whole_weight"], alpha=0.3)
plt.xlabel("Shucked + Viscera + Shell")
plt.ylabel("Whole weight")
plt.title("Does Whole = Sum of parts?")
plt.show()
```

4) Scatter plot of Whole_weight vs sum_parts

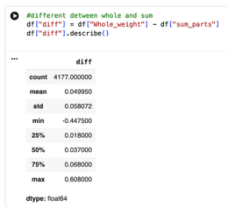


Regression Model

The multilinear regression model
`Whole_weight=0.9366·Shucked_weight+1.1116·Viscera_weight`

`+1.2530·Shell_weight-0.00783`

(c) Look at the difference (Whole – sum_parts)



```
#different between whole and sum
df["diff"] = df["Whole_weight"] - df["sum_parts"]
df["diff"].describe()
```

	diff
count	4177.000000
mean	0.049950
std	0.058072
min	-0.447500
25%	0.018000
50%	0.037000
75%	0.068000
max	0.608000

dtype: float64

Interpretation of coefficients

The coefficients are **not 1 exactly** meaning that the components do not equally contribute.

The shell weight coefficient is also **the largest (1.253)**, indicating increasing weight of shells will lead to an increase in total weights, perhaps because there's a relationship between shell weight and overall abalone size.

Conclusion

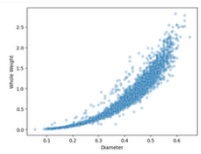
The total value of the predictor sums is **very nearly** equal to 100%. Weight is very strongly (0.995) correlated with the sum of its parts. However, the coefficients in the regression model differ from 1, and the whole weight is slightly greater than the sum of the dried/measured components, indicating **measurement differences and biological variation**.

Q3 Relationship between the whole weight and diameter

Show plot of the relationship

The plot shows a clear **non-linear, upward-curving pattern**. Weight increases slowly for smaller abalones and much more rapidly for larger ones, indicating that a linear model is unlikely to capture the full curvature of the relationship.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.scatterplot(ind["Diameter"], y=df["Whole_weight"], alpha=0.3)
plt.xlabel("Diameter")
plt.ylabel("Whole Weight")
plt.show()
```



(a) A simple linear model

Weight = a *diameter +b
Line Correlation : r = 0.92545 $$

(a) Linear: weight = a*d + b

```
X = df[["Diameter"]]
y = df["Whole_weight"]
model = LinearRegression().fit(X, y)

pred = model.predict(X)
#compute correlation bewteen true and predicted
corr = np.corrcoef(y, pred)[0,1]
```

(b) A quadratic model

Weight=a· diameter +b· diameter^2+c
Quadratic Correlation : r = 0.96268

(b) Quadratic: weight = ad + bd² + c

```
#create quadratic term d^2
df["d2"] = df["Diameter"]**2
#input d and d^2
X = df[["Diameter", "d2"]]
model = LinearRegression().fit(X, y)

pred = model.predict(X)
corr = np.corrcoef(y, pred)[0,1]

print("Quadratic corr =", corr)
```

```
Quadratic corr = 0.962684866053714
```

(c) A cubic model without lower order or constant terms

Weight=a· diameter ^3
Cubic Correlation : r = 0.96307

(c) Cubic : weight = a*d³

```
#create quadratic term d^3
df["d3"] = df["Diameter"]**3
#only d^3
X = df[["d3"]]
model = LinearRegression().fit(X, y)

pred = model.predict(X)
corr = np.corrcoef(y, pred)[0,1]

print("Cubic corr =", corr)
```

```
Cubic corr = 0.9630786738388938
```

(d) An exponential model

log (weight)= a· diameter+b
Exponential **Correlation :** r = 0.96351

(d) Exponential: log(weight) = a*d + b

```
#get log for whole_weight
df["logW"] = np.log(df["Whole_weight"])

#input is diamter
X = df[["Diameter"]]
y_log = df["logW"]

model = LinearRegression().fit(X, y_log)

pred_log = model.predict(X)
corr = np.corrcoef(y_log, pred_log)[0,1]

print("Exponential corr =", corr)
```

```
Exponential corr = 0.9635120791439449
```

Model comparison

Model	Correlation	Performance
Linear	0.92545	Poor/ underfits curvature
Quadratic	0.96268	Good/ captures curvature
Cubic (d³ only)	0.96308	Good/ slightly better than quadratic
Exponential (log-linear)	**0.96351**	**Best/ simplest with highest r**

Final choice

log(Weight) = a *diameter +b

Q4 Logistic Regression

Recoding the target variable

We convert the class label into a binary attribute **Infant (I)** → 1 **Adult (M or F)** → 0

(a) Using Length only

Recoding

```
#tran infant as 1, adult as 0
df["Infant"] = (df["Sex"] == "I").astype(int)
```

Accuracy = 0.77999

Size only gives fair prediction performance. This reflects the fact that infants are usually smaller. But adults can be of different length and thus this feature is an imperfect one for distinguishing them.

(b) Using Whole weight only

(a) Using length only

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

#def helper function to compute accuracy give features
def logistic_accuracy(features):
    #get x feature
    X = df[features]
    y = df["Infant"]

    # increas max_iter for convergence
    model = LogisticRegression(max_iter=1000)
    model.fit(X, y)
    pred = model.predict(X)
    return accuracy_score(y, pred)

print("Length only:", logistic_accuracy(["Length"]))
```

```
Length only: 0.7799856356236533
```

Accuracy= 0.79555

Whole weight has slightly better predictive value than length. This is not surprising, as babies have much smaller total body mass and therefore weight is a more efficient discriminating measure.

(c) Using Rings only

Accuracy = 0.78813

Rings indicate the number of shell rings. Cup age is a very natural predictor given that rings + 1.5 = age and indeed there are fewer rings from noisier pictures as predcited from the children in relation to infants example.

(d) Using Length + Whole weight + Rings (combined model)

Accuracy = 0.81398

This is the **best-performing model** among the four. Each feature contributes complementary information

Length → physical size / Whole weight → body mass / Rings → biological age

Conclusion

Whole weight is the best performing on single attribute, reflecting that infants have much smaller weight comparing to an adult. **The concatenated model (Length + Whole weight + Rings)** achieves the best accuracy at 81.4%, indicating that abalone infant classification is improved when both size and mass as well as relative age-related attributes are used.

Q5 Logistic Regression on the Adult Dataset

Read Data

The Adult dataset was loaded from the UCI repository. Missing values were removed. Categorical attributes were converted using one-hot encoding.

`drop_first=True` to avoid dummy-variable traps

The target variable was recoded as: **Male = 1 / Female = 0**

All one-hot–encoded features were used except the `sex_Male` column, which must be removed to avoid data leakage.

Logistic Regression Model and Accuracy

A logistic regression classifier was trained on the processed dataset:

Accuracy = 0.8081

This means the model correctly predicts an individual's sex about **80.8%** of the time. A convergence warning occurred due to high dimensionality after one-hot encoding; increasing `max_iter` or scaling could improve numerical convergence but does not change the overall conclusion.

(a) Attribute Selection

Removing features without losing accuracy

By iteratively removing features and retraining, several attributes were found to have **minimal impact** on accuracy (≤1% change)
Attributes removable with almost no accuracy drop
`Fnlwgt` / `education-num`/ `capital-gain`/ `capital-loss`/ `native-country`

Reason these can be removed

- `fnlwgt`, `capital-gain` and `capital-loss` show weak direct association with sex, and their information overlaps with other income-related features.
- `education-num` is a numeric encoding of `education`; keeping the categorical education variables and dropping `education-num` causes almost no loss.
- `native-country` is highly imbalanced (mostly "United States"), so it carries little information overall; very rare categories may create unstable coefficients.

(b) Why is relationship-status helpful?

`relationship` is one of the **most predictive** attributes in the dataset.

Examples:

- `"Husband"` → almost exclusively male / `"Wife"` → almost exclusively female
- `"Own-child"` → often female (single mothers)/ `"Not-in-family"` or `"Unmarried"` → more mixed

Gender roles are strongly represented as they correlate with **gender categories** in census data. As a result, the model attaches large weights to such categories, and greatly improves classification accuracy.

(c) Why is country=Holland weighted heavily?

Reason: extremely small sample size

In the Adult dataset:
There is **only one** individual from Holland
That individual happens to be **male**

So the logistic model sees: **Holland⇒Male** 100% becaues a very strong male indicator and predict close to 1

Conclusion

The full logistic regression model achieves ≈ **81% accuracy**. Removing certain weak or redundant attributes maintains accuracy within **1%**.

The most useful features include:

- `relationship`
- hours worked per week
- education level
- several occupation and workclass categories